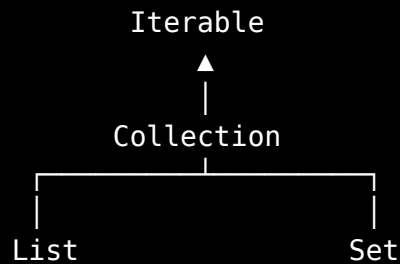


Understand Collection, List, Set, and Iterable Roles (Interview Notes)

Java Collections Hierarchy



Think of it like this:

- **Iterable** → Makes objects iterable using loops.
 - **Collection** → Root interface for most collections.
 - **List** → Ordered collection with duplicates.
 - **Set** → Unique elements only.
-

1. Iterable

What is Iterable?

Iterable is the **top-most interface** that allows an object to be traversed (iterated).

It provides the `iterator()` method, enabling the enhanced for-each loop.

```
Iterable data;
```

Example

```
List list = new ArrayList<>();  
  
list.add("Java");
```

```
list.add("Spring");

for(String s : list){
    System.out.println(s);
}
```

The for-each loop works because List ultimately implements Iterable.

2. Collection

What is Collection?

Collection is the **root interface** of the Java Collections Framework (except Map).

It defines common operations for storing and manipulating groups of objects.

```
Collection c = new ArrayList<>();
```

Common methods:

```
add()
remove()
contains()
size()
clear()
isEmpty()
```

3. List

What is List?

A List is an **ordered collection** that:

- Maintains insertion order
- Allows duplicate elements

- Allows indexing

```
List list = new ArrayList<>();

list.add("Java");
list.add("Java");
list.add("Spring");
```

Output

```
Java
Java
Spring
```

Examples:

- ArrayList
 - LinkedList
 - Vector
-

4. Set

What is Set?

A Set stores **unique elements**.

It automatically removes duplicates.

```
Set set = new HashSet<>();

set.add("Java");
set.add("Java");
set.add("Spring");
```

Output

Java
Spring

Examples:

- HashSet
 - LinkedHashSet
 - TreeSet
-

Collection Hierarchy Example

```
List list = new ArrayList<>();
```

Here,

- ArrayList = Object
- List = Interface
- Collection = Parent Interface
- Iterable = Top Interface

Hierarchy:

```
Iterable
  ↑
Collection
  ↑
List
  ↑
ArrayList
```

Important Interview Points

- **Iterable** provides the ability to traverse elements using an Iterator and enables the enhanced for-each loop.

- **Collection** is the root interface for most collection types and defines common operations like `add()`, `remove()`, and `contains()`.
 - **List** maintains insertion order, allows duplicates, and supports index-based access.
 - **Set** does not allow duplicate elements and is used when uniqueness is required.
 - List and Set are **interfaces**, while `ArrayList`, `LinkedList`, and `HashSet` are their implementations.
 - **Map is not part of the Collection hierarchy** because it stores **key-value pairs**, not individual elements.
-

List vs Set

Feature	List	Set
Order	Maintains order	No guaranteed order (HashSet)
Duplicates	Allowed	Not allowed
Index	Yes	No
Common Implementations	<code>ArrayList</code> , <code>LinkedList</code>	<code>HashSet</code> , <code>TreeSet</code>

Tricky Interview Questions

Q1. What is the difference between Iterable and Collection?

Answer:

- Iterable allows traversal using an iterator.
 - Collection extends Iterable and adds methods for storing and managing data.
-

Q2. Does every Collection implement Iterable?

Answer: Yes.

That's why every collection can be used in a for-each loop.

Q3. Is List a class?

Answer: No.

It is an **interface**.

Q4. Why doesn't Set allow duplicates?

Answer:

A Set is designed to store **only unique elements**.

Q5. Is ArrayList a Collection?

Answer: Yes.

Hierarchy:

```
ArrayList
  ↓
List
  ↓
Collection
  ↓
Iterable
```

Q6. Is Map a Collection?

Answer: No.

Map is a separate interface because it stores **key-value pairs** instead of single elements.

Q7. Can we instantiate Collection or List?

```
Collection c = new Collection(); // ❌
List list = new List();           // ❌
```

Answer: No.

They are interfaces.

Use implementations:

```
Collection c = new ArrayList<>();  
List list = new ArrayList<>();
```

Q8. Why do we write this?

```
List list = new ArrayList<>();
```

instead of

```
ArrayList list = new ArrayList<>();
```

Answer:

Programming to the **interface** makes the code more flexible. You can later replace `ArrayList` with `LinkedList` without changing the rest of your code.

Interview Summary

- **Iterable** enables object traversal using an `Iterator` and the enhanced for-each loop.
- **Collection** is the root interface of the Java Collections Framework (except `Map`) and defines common collection operations.
- **List** is an ordered collection that allows duplicates and index-based access.
- **Set** stores only unique elements and prevents duplicates.
- **Program to interfaces (List, Set, Collection) rather than implementations (ArrayList, HashSet)** for flexible and maintainable code.
- **Map is separate from the Collection hierarchy** because it manages key-value pairs.

Interview One-Liner:

"**Iterable** enables iteration, **Collection** defines common collection operations, **List** stores ordered elements with duplicates, and **Set** stores unique elements without duplicates."